

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Título del Práctica:

Principios SOLID y Uso de IA para Código Seguro

PONDERACIÓN: 3 pts

 **Tiempo estimado: 10 hrs/min**

Índice

1. MARCO FORMATIVO	3
1.1. Valor	3
1.2. Competencia(s)	3
1.3. Habilidad(es) blandas a formar	3
2. Resultado del Aprendizaje	3
2.1. Objetivo SMART	3
3. Enunciado de la Práctica	4
3.1 Descripción del problema a resolver	4
3.2 Alcance de la práctica	4
4. Entregables	5
5. Material de apoyo	5
6. Recursos y herramientas a utilizar	5
7. Cronograma	6
8. Rúbrica de Calificación	6
8.1 Requisitos para optar a la calificación	6
8.2 Resumen de Puntuaciones	7
Detalle de la Calificación	8
Comentarios Generales	9

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	El estudiante entrega un proyecto individual funcional, respetando fechas límite y normas de integridad académica.

1.2. Competencia(s)

Con la elaboración de esta práctica usted adquirirá la siguiente competencia:

Diseñar e implementar una API REST de backend aplicando principios de código limpio y los principios SOLID, demostrando capacidad para construir software escalable, mantenible y de alta calidad utilizando tecnologías modernas de desarrollo, e incorporando el uso responsable de herramientas de Inteligencia Artificial mediante la construcción de prompts efectivos que permitan generar y validar código seguro y limpio.

1.3. Habilidad(es) blandas a formar

La práctica le permitirá desarrollar las siguientes habilidades:

- **Individual:** Autonomía y autogestión en la toma de decisiones técnicas (elección de lenguaje, arquitectura del proyecto).
- **Individual:** Pensamiento crítico para identificar y aplicar correctamente cada principio SOLID en el contexto del problema planteado.
- **Individual:** Comunicación técnica escrita, mediante la redacción de un README claro y bien estructurado en Markdown.
- **Individual:** Uso crítico y responsable de herramientas de Inteligencia Artificial generativa, mediante la construcción de prompts efectivos y la validación de que el código generado sea seguro y limpio.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Implementar una API REST backend para gestionar solicitudes operativas de	El estudiante implementará los 5 endpoints CRUD requeridos, documentará	Mediante el uso libre de lenguajes como JavaScript/Type Script o Python, una base de	Para desarrollar la capacidad de construir software escalable y mantenible,	Antes del 5 de febrero de 2026 a las 23:59 hrs, entregando el repositorio vía UEDI.

una academia ficticia, aplicando principios SOLID y buenas prácticas de código limpio.	los 5 principios SOLID con fragmentos de código real en el README, y registrará al menos 3 prompts de IA utilizados junto con el análisis de la respuesta obtenida.	datos PostgreSQL, un repositorio en la nube y herramientas de IA generativa (ChatGPT, Copilot, Claude, etc.) para apoyar la generación de código.	competencia esencial en el ejercicio profesional de la ingeniería en sistemas.	
--	---	---	--	--

3. Enunciado de la Práctica.

3.1 Descripción del problema a resolver

Se requiere desarrollar un sistema backend para gestionar las **solicitudes operativas** de una academia ficticia. Cada solicitud representa una petición interna que debe ser registrada, consultada, actualizada y eliminada por el sistema.

Cada solicitud operativa tendrá los siguientes atributos:

Campo	Tipo	Descripción
id	Entero o UUID	Identificador único de la solicitud
titulo	String	Nombre descriptivo de la solicitud
area_solicitante	String	Área o departamento que genera la solicitud
prioridad	Entero(1-5)	Nivel de urgencia de la solicitud
costo_estimado	Decimal	Costo Aproximado de atender la solicitud
estado	String	Estado actual: registrada, en_proceso, finalizada

Ejemplo de datos de entrada (JSON):

```
{  
  "titulo": "Adquisición de nuevo servidor",  
  "area_solicitante": "Infraestructura TI",  
  "prioridad": 3,  
  "costo_estimado": 2500.00,  
  "estado": "registrada"  
}
```

3.2 Alcance de la práctica

El sistema deberá soportar las siguientes operaciones mínimas sobre las solicitudes operativas:

- **Obtener** la información de todas las solicitudes operativas.
- **Registrar** una nueva solicitud operativa.
- **Actualizar** completamente la información de una solicitud existente.
- **Eliminar** una solicitud operativa.
- **Actualizar exclusivamente el estado** de una solicitud operativa, sin modificar el resto de sus atributos.

Cada endpoint debe cumplir con los principios SOLID y con las buenas prácticas de diseño REST.

3.3 Uso de Inteligencia Artificial en el desarrollo

Se permite y se fomenta el uso de herramientas de Inteligencia Artificial generativa (ChatGPT, GitHub Copilot, Claude, Gemini, etc.) como apoyo para la generación de código, siempre que el estudiante documente y justifique su uso de forma crítica. Se deben cumplir los siguientes puntos:

- **Diseño de prompts:** redactar prompts claros y específicos que soliciten explícitamente código seguro (validación de entradas, manejo de errores, prevención de inyección SQL) y limpio (nombres descriptivos, responsabilidad única, sin duplicación).
- **Revisión crítica:** analizar el código generado por la IA, identificando y corrigiendo posibles errores, vulnerabilidades o violaciones a los principios SOLID y de código limpio.
- **Registro de evidencia:** documentar al menos 3 prompts utilizados junto con la respuesta obtenida y los ajustes realizados, en un archivo PROMPTS.md o en la sección correspondiente del README.

3.4 Requerimientos técnicos

- La información debe manejarse de forma **persistente** en una base de datos **PostgreSQL**.
- Se recomienda el uso de bases de datos de prueba como **Supabase**, **NeonDB** o un entorno local.
- El lenguaje de programación es de **elección libre** (se recomienda JavaScript/TypeScript o Python).
- El proyecto debe subirse a un **repositorio privado en la nube** (GitHub, GitLab, Bitbucket, etc.) con el formato de nombre: **Prácticas-SA-<<SECCIÓN>>-<<CARNE>>**.
- Crear una carpeta **P1** dentro del repositorio e incluir todos los archivos a entregar.
- Agregar al auxiliar con rol Developer: **Usuario: KevinPozuelos**.

4. Entregables

Describe los productos concretos que se espera que los estudiantes entreguen al finalizar la práctica. Pueden ser prototipos, informes técnicos, documentación, etc.

Tipo	Descripción
Repositorio en la nube	Repositorio privado con el código fuente de la API REST, organizado dentro de la carpeta P1 .
README en Markdown	Documento que incluye: explicación con palabras propias de los 5 principios SOLID y evidencia de su aplicación en el código (archivo/clase, justificación y fragmentos de código reales).
Documentación de Prompts de IA	Archivo PROMPTS.md (o sección en el README) con al menos 3 prompts utilizados para generar o refactorizar código, la respuesta obtenida y los ajustes aplicados para garantizar seguridad y limpieza del código.
Enlace en UEDI	Subir el enlace del repositorio a la plataforma UEDI antes de la fecha límite.

5. Material de apoyo

- [Principios SOLID — Explicación y ejemplos](#)

- [Guía de buenas prácticas REST](#)
- [Documentación de Supabase](#)
- [Documentación de NeonDB](#)
- [Clean Code — Robert C. Martin](#)
- [Guía de Markdown](#)
- OWASP Top 10 — Riesgos de seguridad en aplicaciones web
- Guía de Ingeniería de Prompts (Prompt Engineering) para generación de código seguro

6. Recursos y herramientas a utilizar

- **Software/Hardware:** Computadora con entorno de desarrollo local instalado (Node.js / Python según elección).
- **Base de datos:** PostgreSQL (local, Supabase o NeonDB).
- **Plataformas:** GitHub / GitLab / Bitbucket (repositorio), UEDI (entrega).
- **Lecturas recomendadas:** Clean Code (Robert C. Martin), documentación oficial del lenguaje elegido, guías de diseño REST.
- **Herramientas de IA generativa:** ChatGPT, GitHub Copilot, Claude, Gemini u otra herramienta de IA para apoyo en la generación de código.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Práctica	23/07/2026	23/07/2026
Elaboración	30/07/2026	30/07/2026
Calificación	1/08/2026	1/08/2026

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
------	-------------	----------------

Repositorio	Repositorio privado creado con el formato de nombre correcto y carpeta P1 incluida.	
Acceso al auxiliar	Usuario KevinPozuelos agregado con rol Developer.	
Entrega en UEDI	Enlace del repositorio subido antes del 30 de Julio de 2026 a las 23:59 hrs.	
Trabajo individual	La práctica fue realizada de forma individual, sin copias parciales ni totales.	
Último commit	El último commit fue subido antes de la hora y fecha de entrega.	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades		
Implementación de endpoints CRUD	25	
Aplicación de principios SOLID en el código	20	
Uso efectivo de IA (prompt engineering)	15	
Sub-Total Habilidades	60	

2. Conocimiento		
Documentación README (explicación SOLID)	15	
Documentación y análisis de prompts de IA	15	
Calidad y estructura del código (código limpio)	10	
Sub-Total Conocimiento	40	
TOTAL	100	

Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades					
1.1	Implementación de los 5 endpoints REST (GET, POST, PUT, DELETE, PATCH estado)		Descripción	Todos los endpoints están implementados, funcionales y siguen las convenciones REST correctamente.	Faltan endpoints, no funcionan correctamente o no siguen las convenciones REST.	
			Rango del punteo			
1.2	Aplicación de principios SOLID en el código			El código refleja de forma clara y correcta al menos 4 de los 5 principios SOLID.	El código no aplica los principios SOLID de forma evidente o su aplicación	
1.3	Uso efectivo de IA (prompt engineering)			Los prompts son claros y específicos, solicitan explícitamente código seguro y limpio, y el estudiante revisa críticamente y corrige el código generado por la IA.	Los prompts son genéricos o no solicitan código seguro/limpio, o no se evidencia revisión crítica del código generado por la IA.	

	Sub-Total de Puntos					
2	Conocimientos					
2.1	Documentación README — Explicación de principios SOLID		Descripción	El README explica con palabras propias los 5 principios SOLID e incluye fragmentos de código real y funcional que evidencian su aplicación.	La explicación es incompleta, copiada textualmente de otra fuente, o los fragmentos de código no corresponden al proyecto	
			Rango del punteo			
2.2	Calidad del código (código limpio)			El código es legible, con nombres descriptivos, funciones con responsabilidad única y sin redundancias.	El código es difícil de leer, con nombres poco descriptivos o con lógica mezclada sin estructura	
2.3	Documentación y análisis de prompts de IA			El archivo de prompts incluye al menos 3 prompts reales, la respuesta obtenida y un análisis claro de los ajustes realizados para garantizar seguridad y limpieza del código.	Los prompts documentados son insuficientes, genéricos, o no incluyen análisis de los ajustes realizados al código generado.	
	Sub-Total de Puntos					
	Total					

